

Document number	D3122R0
Date	2024-01-09
Project	Programming Language C++, Library Working Group
Reply-to	Jonathan Wakely <cxx@kayari.org>

Using `[[nodiscard]]` should be Recommended Practice

- - [Introduction](#)
 - [Rationale](#)
 - [Wording](#)
 - [Acknowledgments](#)
 - [References](#)

Introduction

Instead of sprinkling `[[nodiscard]]` in hundreds of places in the library clauses, there should be a single normative recommendation to use it wherever it's appropriate.

Rationale

It's not a good use of time to discuss adding `[[nodiscard]]` to individual function declarations. That requires LEWG time, LWG time, WG21 votes, editorial changes, and reviews to ensure those changes were correctly applied. For example, [P2377R0 `\[\[nodiscard\]\]` in the Standard Library: Clause 23 Iterators Library](#) only covers a single clause and is still nearly ten pages of wording. And it has a mistake in the very first change it proposes to a declaration, which would need to be reviewed and corrected.

What we want is for real implementations to give real warnings to users writing real code. Whether the attribute is literally present on individual declarations in the PDF standard is not important.

[P2377R0](#) argued that it's a good use of committee time to add it to the library because it reminds implementors to use the attribute and improve the quality of their implementations. I feel that a single normative recommendation to use it is a far better use of time than micro-managing how implementations declare individual functions.

Implementers are best placed to decide which functions to add the attribute to for their specific implementation. If a compiler has built-in warnings for side-effect-free expressions, e.g. it can determine that `it == end;` doesn't modify its arguments and so issues a warning, then there is no need for that implementation to explicitly add the `[[nodiscard]]` attribute to every inline `operator==` definition. **We should aim for good quality diagnostics, not worry about how that is achieved.**

The wording below only talks about functions. The valid uses for `nodiscard` on types are less common, and it might be worth discussing individual cases, and adding the attribute to declarations in the standard if implementors need extra guidance there.

Wording

This wording is relative to [N4971](#).

Add a new subclause after 16.4.6.15 [lib.types.movedfrom]:

16.4.6.?? Discarded calls [lib.nodiscard]

Except where shown otherwise, it is unspecified which functions defined in the C++ standard library are marked with a `nodiscard` attribute.

Recommended practice: Implementations should issue a warning for potentially-evaluated discarded-value expressions ([`expr.context`]) where the expression is a call to any of the following:

- `operator==` and `operator<=>` functions.
- const member functions and explicit object member functions that do not directly or indirectly modify objects accessible via their arguments.
[Example: `vector::empty()` const and `unordered_map::bucket_size(size_type)` const do not modify any objects. — end example]
- Non-const member functions that return pointers, references, or iterators, but do not modify the object parameter or the object that the return value refers to.
[Example: `vector::begin()` and `set::find(const key_type&)` are not const so that they can return a mutable iterator, but they do not modify the container or its elements. `basic_string::operator[]` (`size_type`) does not modify the string or its contents. — end example]
- Non-member functions that do not directly or indirectly modify objects accessible via their arguments, other than potentially modifying their non-reference arguments.
[Example: `as_const(T&)` behaves like a cast, it has no side effects. `back_inserter(Container&)` takes a reference to a non-const object but does not modify it. `ranges::next(I)` increments its by-value parameter, which does not modify any other objects. — end example]
- The function call operators of customization point objects ([`customization.point.object`]), unless the return type is `void`.

Acknowledgments

Thanks to Nicolai Josuttis, Hana Dusíková, and Christopher Di Bella for the past work identifying where `nodiscard` is useful.

References

[P0600R1](#), [[`nodiscard`]] in the Library, Nicolai Josuttis, 2017.

[P2351R0](#), Mark all library static cast wrappers as [[`nodiscard`]], Hana Dusíková, 2020.

[P2377R0](#), [[`nodiscard`]] in the Standard Library: Clause 23 Iterators Library, Christopher Di Bella, 2021.

[N4971](#), Working Draft - Programming Languages -- C++, Thomas Köppe, 2023.